

Fachpraktikum 63585

IT-Sicherheit, IT-Forensik und Datenschutz

Tunneln von Nachrichten innerhalb von Messagebroker basierten Protokollen

Gruppenaufgabe

Per Huepenbecker

Carsten Lenzen

Robin Wetzlar



Inhaltsverzeichnis

1. Aufgabenstellung	3
2. Protokolle im Überblick	4
2.1 Corba	4
2.1.1 Funktionsweise	4
2.1.2 Typische Einsatzgebiete	5
2.1.3 Typische Paketgröße	5
2.2 MQTT	6
2.2.1 Funktionsweise	6
2.2.2 Typische Einsatzgebiete	7
2.2.3 Typische Paketgröße	8
2.3 Entscheidung	8
3 Systemdesign	9
3.1 Zielsetzung	9
3.2 Architekturübersicht	9
3.3 Kommunikationsmodell	10
3.4 Protokolldefinition und Datenformat	11
3.5 Segmentierung	13
3.5.1 Vorteile der Segmentierung	13
3.5.1 Nachteile Segmentierung	13
3.6 Einsatz eines TUN-Devices	14
3.7 Sicherheitsbetrachtung	14
4. Implementierung	14
4.1 Entwicklungsumgebung	15
4.2 Aufbau der Software	17
4.3 Installation	18
4.4 TUN-Device-Anbindung	19
4.5 MQTT-Integration	19
4.6 Programmablauf	19
5. Tests und Analyse	20
5.1 Funktionale Tests	20
5.2 Performance und Latenz	20
5.3 Wireshark-Analyse	20
5.4 Fehlerfälle	20

6. Fazit	20
Abbildungsverzeichnis	21
Quellenverzeichnis	22

1. Aufgabenstellung

Es gibt verschiedene Wege und Protokolle, über die sich ein Tunnel realisieren lässt. Allerdings ist es empfehlenswert, einen Tunnel über ein Protokoll zu wählen, das mit einer hohen Wahrscheinlichkeit in einem Netzwerk trotz Firewalling und vielleicht sogar irgendwelcher Proxies verfügbar ist. Das schränkt die Auswahl schon etwas ein.

Vielversprechend und entsprechend häufig genutzt sind

HTTP / HTTPS

DNS

Die wollen wir in diesem Praktikum daher nicht nutzen, das wäre ja langweilig - stattdessen wollen wir einen "Exoten" nutzen: Ein Protokoll, dass einen Message-Broker nutzt, wie

- MQTT
- CORBA

In Ihrer Gruppe wählen Sie eines dieser Protokolle oder ein weiteres broker-basiertes Protokoll aus, dokumentieren das "Warum" (es gibt kein "richtig" und "falsch", aber Ihre Beweggründe sind interessant) und entwickeln, möglichst unter Zuhilfenahme vorhandener Tools / Frameworks / bekannter Lösungen eine Implementierung, um einen Tunnel zu ermöglichen.

Im Idealfall können Sie dann durch diesen Tunnel z.B. mit SSH, OpenVPN oder Wireguard alles weitere durchtunneln, alternativ beschränken Sie sich darauf, daß Ihr Tunnel HTTP(S)-Anfragen durchleitet. Was Sie anbieten, ist eine Design-Entscheidung. Auch dafür sollten Sie die Argumente wieder dokumentieren.

Auch hier geht es um die Überlegungen, nicht um ein "richtig" oder "falsch".

Ihre Aufgaben für die Gruppenarbeit sind also:

1. Erörtern Sie, welches Protokoll Ihr Trägerprotokoll wird.
2. Erörtern Sie, ob Sie allgemein alle TCP oder UDP oder IP-Verbindungen tunneln wollen, oder nur HTTP(S).
3. Implementieren Sie Ihren Tunnel. Dokumentieren Sie Implementationsentscheidungen. Hinweis: Dazu sollten Sie auch die Anforderungen und Lösungsideen gründlich analysieren, der Anspruch ist, daß die Beschreibung zwar abstrakt, aber konkret genug ist, daß auf Ihr Darstellung aufbauend jemand die Implementierung erfolgreich starten kann. Der Teufel liegt bekanntlich im Detail, das vielleicht auch erst bei Tests und der Umsetzung auffällt, das müssen Sie nicht entdecken. Die Ergebnisse sollten sich in Ihrem Paper wiederfinden.
4. Testen Sie Ihren Tunnel gründlich. Dazu gehört auch, den Tunnel mal mit tcpdump, Wireshark o.ä. aufzuzeichnen und zu prüfen, was ein Dritter sieht.
5. Dokumentieren Sie in Ihrer Abgabe, welche Gruppenmitglieder an welchem Teil der Aufgabe mitgewirkt haben, wer für welchen Teil "den Hut auf hatte" und welchen Anteil welches Gruppenmitglied an dem Teil hatte. Im Idealfall sollten im Ergebnis alle Gruppenmitglieder etwa gleich viel geleistet haben. Die Aufgaben bearbeiten Sie in Gruppen von bis zu vier Personen. (Jedes Schild hat eine Geschichte, so auch der Hinweis, daß eine Personengruppe mindestens zwei Personen umfasst.) Das Paper (also der gedanklich-wissenschaftliche Hintergrund) geben Sie als .pdf-Datei über Moodle ab, die Implementierung im (ggf. compilierbaren) Quellcode als .tar.gz.

Sie sind frei in der Wahl der Programmiersprache(n), Sie dürfen auch auf z.B. den Code von iodine oder anderen Frameworks aufsetzen, wenn das für Sie sinnvoll erscheint. Das sind Design-Entscheidungen, die sie in Ihrem Paper diskutieren und erläutern sollten.

2. Protokolle im Überblick

Um eine Entscheidungsgrundlage zu gewährleisten werden im ersten Schritt die beiden vorgeschlagenen Protokolle mit Messagebrokercharakter in Kürze dargestellt.

Dabei werden die wesentlichen Funktionsweisen, typische Anwendungsbereiche und markante Merkmale der jeweiligen Protokolle dargestellt.

Im weiteren Verlauf wird die letztendlich getroffenen Entscheidung, welches der beiden eingangs genannten Protokolle für die weitere Bearbeitung der Aufgabenstellung Relevanz findet. Diese Entscheidung wird anhand der aufgezeigten Charakteristika begründet.

2.1 Corba

CORBA (Common Object Request Broker Architecture) ist kein einzelnes Protokoll im herkömmlichen Sinne, sondern kann als Standard für eine Middlewarearchitektur, welche eine Kommunikation zwischen verteilten Softwarekomponenten ermöglicht, verstanden werden. Diese ist dabei unabhängig von der jeweilig gewählten Programmiersprache, Betriebssystem oder Hardwareplattform. Das spezifische Netzwerkprotokoll, das CORBA für die Interoperabilität verwendet, heißt Internet Inter-ORB Protocol (IIOP) und basiert auf TCP/IP. ist eine objektorientierte Middleware für verteilte heterogene Systeme, die von der Object Management Group entworfen und spezifiziert wurde [1,2,3]

2.1.1 Funktionsweise

Das Kernstück der CORBA-Architektur ist der Object Request Broker (ORB). Der ORB ist verantwortlich für die Vermittlung aller Anfragen und Antworten zwischen Clients (Nutzern eines Dienstes) und Servern (Anbietern eines Dienstes).

Der Prozess der Kommunikation funktioniert in mehreren Schritten:

1. Schnittstellendefinition (IDL):
Die Schnittstelle des Server-Objekts wird mithilfe der Interface Definition Language (IDL) getreu den Spezifikationen der Object Management Group festgelegt. Diese sind sprachneutral beschrieben und zeigen genau auf, welche Operationen das Objekt bereitstellt und welche Parameter benötigt werden.
2. Generierung von Stubs und Skeletons:
Aus der IDL-Definition werden sprachspezifische Codefragmente generiert:
 1. Stubs auf der Client-Seite:
Sie verbergen die Netzwerkkommunikation vor dem Client, sodass sich der Aufruf einer entfernten Methode wie der Aufruf einer lokalen Methode anfühlt.
 2. Skeletons auf der Server-Seite:
Sie nehmen die Anfragen entgegen, entpacken die Daten und rufen die tatsächliche Implementierung der Server-Logik auf.
3. Objektreferenz und Naming Service:
Der Server erstellt das Objekt und registriert es bei einem Naming Service. Der Client kann dann über diesen Dienst eine eindeutige Objektreferenz (ähnlich einer Adresse) für das benötigte Objekt erhalten.
4. Der ORB als Vermittler:
Wenn der Client eine Methode aufruft (über den Stub), fängt der lokale ORB diese Anfrage ab. Er verwendet die Objektreferenz, um das entfernte Server-Objekt im Netzwerk zu lokalisieren.
5. Datenübertragung mit IIOP:
Die Anfrage (einschließlich Methodennamen und Parameter) wird vom Client-ORB serialisiert (gemarkt)¹ und über das Netzwerk mithilfe des IIOP an den Server-ORB gesendet. IIOP verwendet hierfür TCP/IP.

¹ Marshalling ist der Prozess der Transformation von Typen, wenn sie zwischen verwaltetem und systemeigenem Code wechseln müssen.[4]

6. Ausführung und Antwort:

Der Server-ORB empfängt die Daten, leitet sie an das Skeleton weiter, das die tatsächliche Methode auf dem Server-Objekt ausführt. Die Antwort wird dann auf dem gleichen Weg zurück an den Client gesendet.

Zusammenfassend ermöglicht CORBA die Transparenz der Lokalität, sodass Entwickler verteilte Anwendungen schreiben können und dabei die Annahme treffen können, dass sich alle Objekte verhalten, als ob sie sich im selben Speicherraum befinden. [1,2,3]

2.1.2 Typische Einsatzgebiete

Das CORBA-Protokoll wird heutzutage vor allem in spezifischen Bereichen und für die Integration von älteren Systemen (Altsystemen) genutzt. Obwohl es in neuen Softwareentwicklungen oft durch modernere Alternativen wie Webservices oder Microservices ersetzt wird, findet es in folgenden Bereichen weiterhin Anwendung:

- **Altsysteme (Legacy-Systeme):**

Eine der Hauptanwendungen ist die Integration bestehender, älterer Software, die auf CORBA basiert. Unternehmen, die in diese Systeme investiert haben, nutzen CORBA weiterhin, um die Funktionalität aufrechtzuerhalten und mit neuen Systemen zu verbinden.

- **Echtzeit- und eingebettete Systeme:**

System in der Luft- und Raumfahrt, wie beispielsweise die Steuerung von Flugzeugfunkgeräten oder Systeme für das Hubble-Weltraumteleskop, nutzten CORBA aufgrund seiner Zuverlässigkeit und Interoperabilität in Echtzeitumgebungen.²

- **Große Unternehmensinfrastrukturen:**

In komplexen IT-Landschaften großer Unternehmen, oft im Finanz- oder Versicherungssektor, wird CORBA weiterhin zur Kommunikation zwischen verschiedenen, heterogenen Softwarekomponenten verwendet.

- **Industrielle Automatisierung und Steuerungssysteme:**

In manchen industriellen Umgebungen wird CORBA genutzt, um verschiedene Maschinen und Steuerungssysteme plattform- und sprachunabhängig zu verbinden.

Generell lässt sich sagen, dass CORBA dort genutzt wird, wo die Migration zu neueren Technologien zu aufwändig oder riskant wäre oder wo seine spezifischen Eigenschaften, wie die garantierte Interoperabilität zwischen verschiedenen Programmiersprachen und Plattformen, nach wie vor benötigt werden. [5,6,7,8]

Da die Echtzeitfähigkeit für die Aufgabenstellung keine Relevanz darstellt, wird an dieser Stelle nicht weiter auf die Thematik und Funktionsweise des RT-Corba eingegangen.

2.1.3 Typische Paketgröße

Es gibt keine feste "typische" Größe für CORBA-Datenpakete, da diese stark von der übertragenen Datenmenge in einem Methodenaufruf abhängt. Die Paketgröße wird im Wesentlichen durch die Nutzdaten der Anwendung bestimmt, wie zum Beispiel durch die Größe von Datenstrukturen, Arrays, Sequenzen oder Dateien, die gesendet werden. CORBA verwendet das General Inter-ORB Protocol (GIOP) bzw. dessen TCP/IP-Implementierung Internet Inter-ORB Protocol (IIOP) für die Kommunikation.

² Mit Hilfe einer erweiterten Implementierung namens RT-CORBA, die Systemressourcen (CPU, Netzwerk, Threads) basierend auf zugewiesenen Prioritäten vorhersagbar verwaltet und steuert.[9]

Die Paketgröße der IIOP-Nachrichten steht dabei in direkter Abhängigkeit von verschiedenen Faktoren.

- **Anwendungsdaten:**

Dies ist der Hauptfaktor. Ein Aufruf, der nur einen einfachen Integer überträgt, erzeugt ein sehr kleines Paket. Ein Aufruf, der ein großes Array oder eine komplexe Datenstruktur mit umfangreichen Inhalten überträgt, erzeugt entsprechend größere Nachrichten.

- **Netzwerk-MTU (Maximum Transmission Unit):**

Auf der Netzwerkebene, insbesondere bei Ethernet über TCP/IP, liegt die maximale Paketgröße (MTU) üblicherweise bei etwa 1500 Bytes. Größere IIOP-Nachrichten werden automatisch vom zugrunde liegenden TCP/IP-Stack oder dem ORB selbst in mehrere kleinere IP-Pakete (Fragmentierung) zerlegt, um über das Netzwerk gesendet zu werden.

- **ORB-Konfiguration (Fragmentierung):**

Viele ORB-Implementierungen verfügen über eine Konfigurationseinstellung für die maximale Nachrichtengröße, bevor eine Fragmentierung auf GIOP-Ebene stattfindet. Der Standardwert für diese Fragmentgröße ist oft 1024 Bytes (z. B. bei IBM oder Oracle ORBs), kann aber vom Entwickler angepasst werden. Nachrichten, die größer als dieser Wert sind, werden in Fragmente aufgeteilt.

- **Header-Overhead:**

Jedes IIOP/GIOP-Paket hat einen festen Header von 12 Bytes, plus zusätzliche Overheads durch TCP (ca. 20-32 Bytes) und IP (ca. 20 Bytes).

Abschließend lässt sich sagen, dass es keine typische Größe für CORBA-Pakete. Kleine Pakete können nur wenige Dutzend Bytes groß sein, während große Datenübertragungen leicht mehrere Megabyte erreichen können und teils fragmentiert über das Netzwerk gesendet werden. Die Paketgröße steht im direkten Zusammenhang zur Datenmenge, die von der jeweiligen Anwendung über die definierte IDL-Schnittstelle gesendet werden.
[1,2,10]

2.2 MQTT

MQTT (Message Queuing Telemetry Transport) ist ein schlankes, offenes Nachrichtenprotokoll, das speziell für die Machine-to-Machine (M2M)-Kommunikation im Internet of things³ (IoT) entwickelt wurde. Es ist darauf ausgelegt, Daten effizient in Umgebungen mit begrenzter Bandbreite, hoher Latenz oder unzuverlässigen Netzwerkverbindungen zu übertragen. [11, 12]

2.2.1 Funktionsweise

Im Gegensatz zum traditionellen Client-Server-Modell (wie HTTP mit Anfrage / Antwort) basiert MQTT auf dem Publish / Subscribe-Modell (Veröffentlichen / Abonnieren). Die Kommunikation wird über eine zentrale Komponente, den sogenannten MQTT-Broker, abgewickelt.

Die Architektur umfasst drei Hauptbestandteile:

- **Publisher (Veröffentlicher):**

Ein Gerät (Client), das Daten sendet, indem es Nachrichten zu einem bestimmten Thema (Topic) an den Broker "veröffentlicht".

- **Subscriber (Abonnent):**

Ein Gerät (Client), das Nachrichten empfängt, indem es bestimmte Themen (Topics) beim Broker "abonniert".

- **Broker:**

Die zentrale Instanz, die als Vermittler fungiert. Er nimmt Nachrichten von Publishern entgegen, filtert sie nach Themen und leitet sie an alle interessierten Subscriber weiter.

³ Auch *internet der Dinge* genannt

Das System entkoppelt Sender und Empfänger räumlich und zeitlich. Das bedeutet, dass der Publisher nicht wissen muss, wer die Daten empfängt. Seitens Subscriber ist ein analoges Verhalten erkennbar, da dieser nicht wissen muss, woher die Daten stammen. [11,13]

Vereinfacht lässt sich sagen, dass in einem MQTT-Netz alle Themen (Topics), von einem Publisher gespeist und von einem Subscriber gelesen werden.

2.2.2 Typische Einsatzgebiete

MQTT wird heute als De-facto-Standard für die Datenübertragung im Internet der Dinge angesehen und findet aufgrund seiner Effizienz, Leichtgewichtigkeit und des Publish/Subscribe-Modells in zahlreichen Bereichen Anwendung. Diese erstrecken sich typischerweise auf die folgenden Anwendungsbereiche.

- **Internet der Dinge (IoT) allgemein:**
MQTT ist das führende Protokoll für die Kommunikation zwischen intelligenten Geräten, Sensoren und Aktoren. Es wurde speziell für diese Zwecke entwickelt, um Daten effizient zu übertragen.
- **Smart Home & Gebäudeautomatisierung:**
In intelligenten Häusern wird MQTT genutzt, um verschiedenste Geräte wie Beleuchtungssysteme, Thermostate, Türschlösser und Sicherheitssysteme zu vernetzen und zentral zu steuern, oft in Verbindung mit Plattformen wie Home Assistant oder OpenHAB.
- **Industrielle Automatisierung (IIoT):**
In der Fertigungsindustrie spielt MQTT eine zentrale Rolle für die Kommunikation zwischen Maschinen und Systemen. Es wird für Zustandsüberwachung (z.B. Motorstatus), vorausschauende Wartung, Energiemonitoring und die Vernetzung von Produktionsanlagen genutzt.
- **Telemetrie und Sensornetzwerke**
Ursprünglich für die Öl- und Gasindustrie zur Kommunikation über Satellit entwickelt, eignet sich MQTT perfekt für die Übertragung von Telemetriedaten (Messdaten) von entfernten oder schwer zugänglichen Sensoren.
- **Standortverfolgung und Logistik**
Systeme zur Diebstahlortung oder Standortüberwachung in der Logistik nutzen MQTT, um Positionsdaten effizient zu übermitteln.
- **Gesundheitswesen (Wearables)**
Intelligente Wearables und medizinische Sensoren, die Gesundheitsdaten sammeln und an Backend-Systeme übermitteln, verwenden häufig MQTT.
- **Cloud-Plattformen**
Große Cloud-Anbieter wie Amazon Web Services (AWS IoT), Microsoft Azure und Google Cloud bieten native Unterstützung für MQTT, um die Anbindung und Verwaltung Tausender von IoT-Geräten zu ermöglichen. [11,14,15,16]

Die Beliebtheit von MQTT in diesen Bereichen liegt in seinen Eigenschaften begründet:

- **Leichtgewichtig:**
Das Protokoll hat einen geringen Daten-Overhead und benötigt nur wenig Bandbreite, was ideal für ressourcenbeschränkte Geräte ist.
- **Effizientes Design:**
Das Publish/Subscribe-Modell entkoppelt Sender und Empfänger, was eine skalierbare und flexible Architektur ermöglicht.
- **Zuverlässigkeit:**
Es bietet verschiedene Quality-of-Service (QoS)-Stufen, um sicherzustellen, dass Nachrichten je nach Anforderung zuverlässig zugestellt werden.

2.2.3 Typische Paketgröße

Ähnlich wie bei CORBA oder HTTP gibt es keine einzelne, feste Größe für MQTT-Datenpakete. Die Paketgröße und die Menge der tatsächlichen Nutzdaten (Payload) zeigt sich anhängig von der Datenmenge, die von der jeweiligen Anwendung übertragen werden. MQTT ist jedoch darauf ausgelegt, extrem leichtgewichtig und effizient zu sein, um den Protokoll-Overhead zu minimieren.

Dabei zeigt sich der Aufbau eines MQTT-Pakets im klassischen Sinne nach dem folgenden Schema. Dabei besteht ein MQTT-Nachrichtenpaket aus den folgenden drei Teilen:

- **Fixed Header (Fester Header):**
Immer vorhanden und nur zwei Bytes groß. Er enthält grundlegende Informationen wie den Pakettyp (z.B. CONNECT, PUBLISH, SUBSCRIBE), QoS-Level und Flags.
- **Variable Header (Variabler Header):**
Seine Größe variiert je nach Pakettyp. Bei einer PUBLISH-Nachricht enthält er beispielsweise die Länge und den Namen des Topics.
- **Payload (Nutzdaten):**
Dies sind die eigentlichen Anwendungsdaten, die gesendet werden (z.B. der Sensorwert eines Temperatursensors wie {"temp": 21.5}). Dabei kann die Größe der Payload von einem einzigen Byte bis hin zu mehreren Megabyte reichen.

Die Gesamtgröße eines MQTT-Pakets wird durch die Summe dieser Teile bestimmt.

- **Mindestgröße:**
Das kleinste mögliche PUBLISH-Paket (ohne Payload, mit sehr kurzem Topic) ist nur wenige Bytes groß (oft unter 10 Bytes Overhead).
- **Typische Nutzung:**
In IoT-Anwendungen, wo oft kleine Sensorwerte oder Statusinformationen (wie ON oder OFF oder ein JSON-Objekt mit wenigen Werten) gesendet werden, sind Pakete von wenigen Dutzend Bytes bis zu einigen Kilobytes am häufigsten.
- **Maximale Größe:**
Die MQTT-Spezifikation erlaubt in der Version 3.1.1 eine maximale Paketgröße von 256 Megabyte (inklusive Header und Payload).
- **Netzwerk-Limitierung:**
Wie bei jedem TCP/IP-Protokoll werden auch große MQTT-Pakete auf der Netzwerkebene durch die Maximum Transmission Unit (MTU) – typischerweise 1500 Bytes bei Ethernet – fragmentiert und über mehrere IP-Pakete gesendet.

Zusammenfassend lässt sich sagen, dass MQTT-Pakete aufgrund ihres minimalen Overheads in der Regel sehr klein sind, die tatsächliche Größe aber in direkter Abhängigkeit der individuellen Anforderung steht.

2.3 Entscheidung

Auf Grund der dargestellten Informationslage haben wir uns innerhalb der Gruppenarbeit auf die Nutzung des MQTT Protokolls geeinigt.

Die Begründung liegt dabei hauptsächlich in dem breiteren Anwendungsspektrum gepaart mit dem gleichzeitig effizient gestalteten Nachrichtenaufbau.

Zeitgleich dient die prominente Verbreitung des MQTT-Protokolls innerhalb der IoT-Thematik vermehrte Berührungspunkte im alltäglichen Leben

Der stetig steigende Trend beim Einsatz sogenannter Smarthome-Komponenten in Verbindung mit den verschiedensten Steuerungszentralen, welche zum Beispiel auf OpenSource-Plattformen wie Home-assistant⁴ oder IO-Broker⁵ basieren, birgt weiteres Potential zur gesteigerten Verbreitung des MQTT-Protokolls und damit größeren Einfluss im privaten Bereich.

⁴ <https://www.home-assistant.io/>

⁵ <https://www.iobroker.net/?lang=de>

3 Systemdesign

Um die prinzipielle Funktionsweise, welche in Kapitel 1 laut Aufgabenstellung gefordert wird, zu erhalten, wird im ersten Schritt ein Aufbau nach Minimalkonfiguration angestrebt. Diese wird im weiteren Verlauf genauer erläutert.

3.1 Zielsetzung

Die angestrebte Funktion zeigt sich durch einen erfolgreichen Datenaustausch zwischen verschiedener Maschinen. Dabei sollen die Clients über eine bestehende Netzwerktopologie einen Informationsfluss getreu dem MQTT-Protokoll aufbauen und miteinander kommunizieren.

Eine stark vereinfachte Darstellung des angestrebten Informationsaustauschs ist in Abbildung 1 zu sehen. Dort werden die MQTT-Clients über eine entsprechende Datenverbindung mit dem MQTT-Broker verknüpft. Dieser kontrolliert den Datenfluss anhand der vorhandenen Topics und den zugehörigen Subscribern und Publishern.

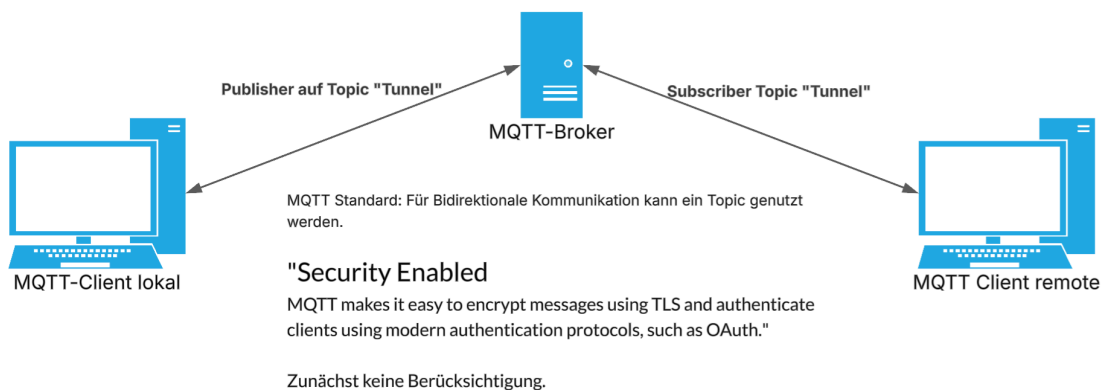


Abbildung 1 Übersicht MQTT-Clients und MQTT-Broker

3.2 Architekturübersicht

Bedingt durch die Entwicklung einer eigenen Anwendung zum Tunneln von Daten via MQTT bietet sich der Einsatz einer TUN-Device an. Ein TUN-Gerät ist eine spezifische Art von virtuellem Netzwerkgerät im Linux-Kernel, das zur Implementierung von Netzwerk-Tunneln verwendet wird, insbesondere für VPN-Dienste (Virtual Private Network). Es arbeitet auf der Netzwerkschicht (Schicht 3) und verarbeitet IP-Pakete. [19]

Über diese TUN-Devices lässt sich ein virtueller Netzwerkunnel zwischen einzelnen Teilnehmern aufbauen. Dabei läuft die Kommunikation innerhalb des Tunnels unabhängig von der Technologie der physischen Umgebung.

Der resultierende Funktionsablauf ist in Abbildung 2 dargestellt. Durch den Einsatz einer TUN-Device wird der TCP-Traffic der Applikation mit dem MQTT Gateway verknüpft. Über die Publish / Subscribe Methodik wird mittels des gemeinsamen Topics ein MQTT-Netzwerk zwischen den Clients hergestellt. Nachdem erfolgreichen Spannen des MQTT-Netzwerks beginnt der Informationsaustausch. Dazu wird der TCP-Datenstrom als MQTT-Payload verpackt und auf das entsprechende Topic veröffentlicht. Durch das Subscriben des Topics durch die Gegenstelle, empfängt

dieser Client den Datenstrom. Ein Socket stellt eine Verbindung zu einer virtuellen TCP-Schnittstelle her, die wiederum als TUN-Device mit der Empfangsapplikation kommuniziert.

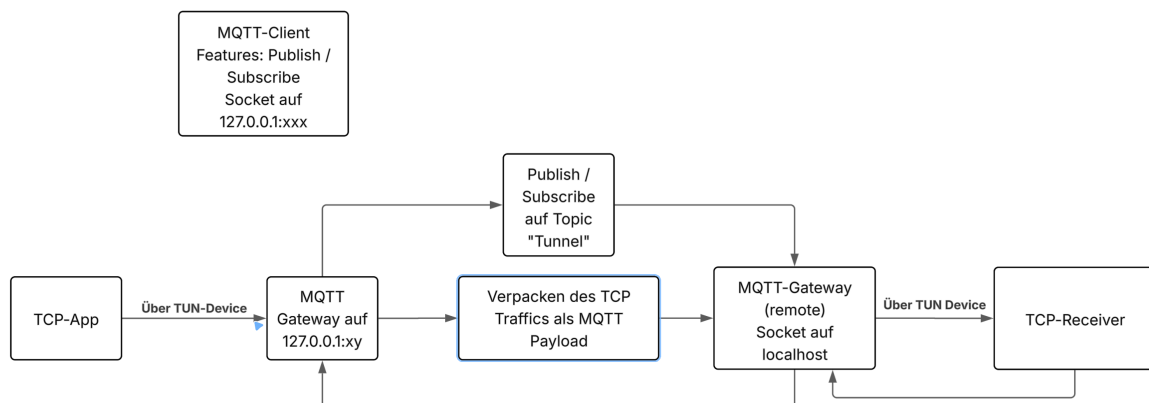


Abbildung 2 Darstellung des angestrebten Funktionsablaufs

3.3 Kommunikationsmodell

Für die bidirektionale Kommunikation zwischen Tunnel-Client und Tunnel-Server werden bewusst zwei getrennte MQTT-Topics verwendet:

- **tunnel/client_to_server**
- **tunnel/server_to_client**

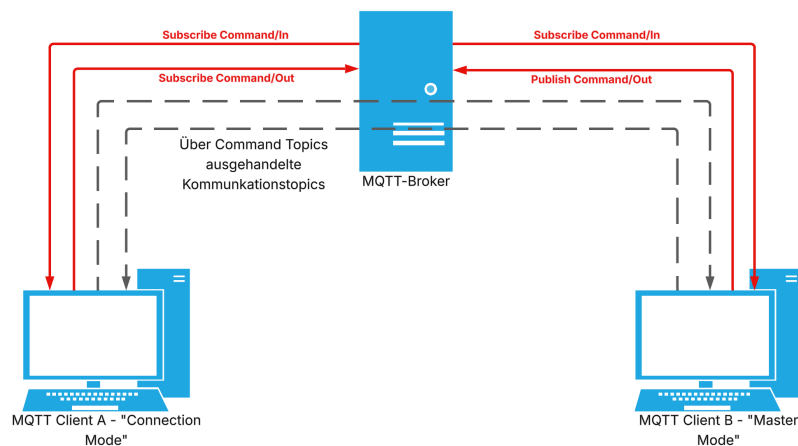


Abbildung 3 Übersicht zum Informationsfluss der MQTT-Topics

Diese Entscheidung wurde aus folgenden Gründen getroffen:

- sie verhindert zuverlässig, dass ein Client seine eigenen Nachrichten wieder empfängt
- viele verbreitete MQTT-Broker unterstützen das MQTT-5-Flag `no_local` nicht oder verarbeiten es inkonsistent
- dadurch ist die Lösung auch mit **MQTT 3.1.1** vollständig kompatibel
- das Verhalten bleibt deterministisch und unabhängig vom Broker
- die Implementierung wird robuster gegenüber unterschiedlichen Broker-Implementierungen (Mosquitto, EMQX, HiveMQ, VerneMQ usw.)

Durch diese klare Trennung der Datenrichtungen wird ein stabiler und vorhersagbarer Tunnelbetrieb gewährleistet.

Darüber hinaus steigert der Einsatz von unterschiedlichen Topics die Übersichtlichkeit im Hinblick auf eine Datenstromanalyse. Diese Datenstromanalyse erfolgt über etablierte Softwarelösungen wie TCPDump⁶ oder Wireshark⁷.

3.4 Protokolldefinition und Datenformat

MQTT selbst auf TCP/IP aufsetzt, handelt es sich beim Tunneling technisch um eine Kapselung von TCP-Daten innerhalb der Anwendungsschicht von MQTT. [20, 21]

- **Rollen:**

Ein lokaler Client nimmt TCP-Verbindungen entgegen, zerlegt den Datenstrom in kleinere Pakete und sendet diese als MQTT-Nachrichten an einen MQTT-Broker. Ein entfernter Client abonniert, empfängt und setzt diese Nachrichten zu einem TCP-Datenstrom zusammen.

- **Topic-Struktur:**

Zur Steuerung und Trennung von Verbindungen können spezifische Topics, z. B. tunnel/device_id/data für den Nutzdatentransport und tunnel/device_id/control für den Verbindungsaufbau oder -abbruch, genutzt werden.

- **Quality of Service (QoS):**

Da TCP eine verlustfreie Übertragung garantiert, wird beim Tunneling meist QoS 1 (mindestens einmal) oder QoS 2 (genau einmal) verwendet, um die Integrität des Datenstroms über den MQTT-Broker hinweg sicherzustellen.

Über den QoS wird die Zuverlässigkeit der Nachrichtenübertragung zwischen Sender (Publisher), Broker und Empfänger (Subscriber) innerhalb von MQTT definiert. Es gibt drei Stufen, die jeweils einen Kompromiss zwischen Zustellgarantie und Netzwerkbelastung darstellen. [23]

Die drei QoS-Level gliedern sich dabei wie folgt. [23]

Level	Bezeichnung	Garantie	Funktionsweise
QoS 0	At most once	Maximal einmal	"Fire and forget". Die Nachricht wird ohne Bestätigung gesendet; sie kann bei Verbindungsproblemen verloren gehen.
QoS 1	At least once	Mindestens einmal	Der Sender speichert die Nachricht, bis er ein PUBACK vom Empfänger erhält. Bei Ausbleiben wird erneut gesendet, was Duplikate erzeugen kann.
QoS 2	Exactly once	Genau einmal	Höchste Stufe durch einen vierstufigen Handshake (PUBLISH, PUBREC, PUBREL, PUBCOMP). Verhindert sowohl Verlust als auch Duplikate.

- **QoS-Aushandlung:**

Der Broker verwendet für die Weiterleitung an den Subscriber das niedrigere Level aus dem Angebot des Publishers und der Anforderung des Subscribers.

⁶ <https://www.tcpdump.org/>

⁷ <https://www.wireshark.org/>

- **Persistent Sessions:**

Um Nachrichten für Offline-Clients (nur bei QoS 1 & 2) zwischenspeichern, muss die Session als "persistent" konfiguriert sein.

- **Performance:**

QoS 0 ist am schnellsten und benötigt die geringste Bandbreite. QoS 2 ist aufgrund des Overheads am langsamsten und ressourcenintensivsten.

Eine Gegenüberstellung der einzelnen QoS-Levels zeigt demnach folgende Anwendungsbereiche.

- **QoS 0:**

Ideal für hochfrequente Sensordaten (z.B. Temperatur alle 100ms), bei denen der Verlust eines Einzelwerts als unkritisch bewertet wird.

- **QoS 1:**

Der Standard für die meisten IoT-Anwendungen, bei denen Daten ankommen müssen, aber die Applikation mit Duplikaten umgehen kann.

- **QoS 2:**

Erforderlich für kritische Transaktionen, wie z. B. Bestellprozesse oder Alarmer, bei denen eine Mehrfachausführung Schäden verursachen würde.

MQTT ist datenagnostisch, das heißt, es definiert kein festes Kodierungsschema für die Nutzlast (Payload). Beim Tunneling wird die Payload wie folgt strukturiert. [20, 21]

- **Payload-Inhalt:**

Die TCP-Segmente werden als binärer Datenstrom (Byte Stream) in das PUBLISH-Paket von MQTT eingebettet.

- **Header-Informationen:**

Da MQTT-Pakete keine native Sequenzierung für Anwendungsdaten besitzen, fügen Tunneling-Tools oft einen eigenen kleinen Header innerhalb der Payload hinzu, um die Reihenfolge der TCP-Datenpakete beim Empfänger sicherzustellen.

- **Payload Format Indicator:**

In modernen Implementierungen (MQTT v5.0) kann das Feld „Payload Format Indicator“ auf 0 gesetzt werden, um explizit einen nicht spezifizierten Byte-Stream zu kennzeichnen.

Der Payload Format Indicator ist eine neue Eigenschaft, die in MQTT 5.0 eingeführt wurde, um das Format der Payload in MQTT-Paketen anzuzeigen. Das Format der Nutzlast in CONNECT-, SUBSCRIBE- und UNSUBSCRIBE-Paketen ist jedoch festgelegt, so dass in der Praxis nur PUBLISH- und CONNECT-Pakete das Payload-Format deklarieren müssen.

Dieses Flag wird dem Empfänger eine Überprüfung des Nutzlastformats ermöglicht, ohne dabei den spezifischen Inhalt parsen zu müssen. Beispielsweise kann der Server überprüfen, ob die Nutzlast eine gültige UTF-8-Zeichenfolge ist. Dadurch soll eine Verteilung falsch formatierter Anwendungsnachrichten an Abonnenten vermieden werden.

Unter Abwägung der vorgangsbedingten Serverbelastung und den potentiellen Vorteilen, ist dies im Regelfall eine optionale Einstellung.

Wie bereits unter 2.2.2 geschildert, gilt der schlanke Protokollaufbau als einer der größten Vorteile von MQTT. Durch die damit verbundene Einsparung an Overhead ist ein Betrieb mit weniger leistungsschwächeren Hardware möglich.

3.5 Segmentierung

Trotz der enormen Flexibilität im Bezug auf die Nachrichtengröße innerhalb der Spezifikationen des MQTT-Protokolls empfiehlt es sich beim Tunneln von großen Datenmengen diese in kleinere Pakete aufzuteilen. [18] Diese sogenannte Segmentierung bietet mehrere Vor- aber auch Nachteile.

3.5.1 Vorteile der Segmentierung

- **Ressourcenbeschränkung**

MQTT dient oftmals als Kommunikationsprotokoll für Sensoren und Aktuatoren. Diese Geräte sind meistens mit ressourcenbeschränkten Komponenten wie Mikrocontrollern ausgestattet, die auf Grund mangelnden Zwischenspeichers mit großen Nachrichten nur bedingt umgehen können. Ein Überschreiten der hardwarebedingten Grenzen kann zu Verbindungsabbrüchen oder dem Verwerfen der Nachricht durch den Broker führen. Darüber hinaus kann eine Überlastung des Speichers dazu führen, dass die jeweiligen Komponenten zu viel Zeit für das Verarbeiten der Nachricht benötigen, wodurch das Abarbeiten der ordinär zugeordneten Hauptaufgabe beeinträchtigt oder zumindest verzögert erfolgt.

- **Netzwerkstabilität**

Große, einzelne Übertragungen sind anfälliger für Netzwerkinstabilitäten. Angenommen es tritt während der Übertragung einer großen Nachricht ein Verbindungsabbruch auf. In diesem Falle müsste die gesamte Nachricht erneut versandt werden, da MQTT keinen Mechanismus zur Fortsetzung unterbrochener Datenströme bietet. Im Falle von kleineren Segmenten, muss lediglich der fehlerhaft übertragene Datenframe erneut gesendet werden.

3.5.1 Nachteile Segmentierung

Die Nachteile der Datensegmentierung umfassen eine Reihe von Herausforderungen, die von hohen Kosten und größerem Aufwand in der Implementierung bis hin zu mangelhafter Datenqualität reichen.

- **Kosten und Ressourcenbedarf**

Der Begriff Kosten wird im weiteren mit der Belastung der Kapazität des Übertragungskanal gleichgesetzt. Erhöhte Kosten zeichnen sich für den Endanwender durch eine verminderte Übertragungsgeschwindigkeit relevanter Daten ab.

Mit jedem Datensegment, welches versendet wird, müssen zusätzliche Informationen an das versandte Element angehängt werden. Diese bieten für den Nutzer keinen Mehrwert. Ein sogenannter Overhead entsteht, welcher für die erhöhten Kosten verantwortlich ist. Dieser Overhead enthält unter anderem Informationen zur originalen Nachricht, Segmentnummer oder Details zu nachfolgenden Paketen, wie z.B. letztes Segment der Ursprungsnachricht.

Wird die originale Nachricht zu fein unterteilt, steigt die Anzahl der Segmente und damit die Menge des damit verbundenen Overheads proportional. Allgemein wird dieses Phänomen als Übersegmentierung bezeichnet.

Ist die Segmentierung zu grob, treten möglicherweise die gewünschten Effekte nicht auf und es wird lediglich ein erhöhter Datenstrom verzeichnet.

- **Planung und Umsetzung**

Wie bereits erwähnt sind sowohl die zu grobe, wie auch eine zu feine Segmentierung nicht wünschenswert. Demnach kann der hinterlegte Aufteilungsprozess sehr komplex sein und erfordert sorgfältige Planung.

- **Datenqualität**

Wie bereits im Vorfeld erwähnt, bietet MQTT keinen eigenen Mechanismus zur Fortsetzung unterbrochener Datenströme. Im Falle von fehlerhaften Segmenten können demnach vereinzelte Nachrichteninhalte verloren gehen. Wird dies nicht von den entsprechenden Anwendungen erkannt oder gar korrigiert, endet die Nachrichtenübertragung in einer verminderten Datenqualität.

3.6 Einsatz eines TUN-Devices

Bei einem TUN-Device handelt es sich um eine virtuelle Netzwerkschnittstelle und kann vereinfacht als Punkt-zu-Punkt- oder Ethernet-Gerät betrachtet werden. Die Besonderheit liegt darin, dass anstelle einer Datenübertragung per physischen Medium, die Datenverbindung virtuell über eine eigene Anwendung aufgebaut wird. [19]

Der Kernel behandelt die Datenpakete der TUN-Device genauso, als würden diese von einem echten physischen Gerät abstammen. Die TUN-Device übernimmt dabei sämtliche Netzwerkrelevanten Aufgaben wie z.B. auch die Segmentierung des ankommenden Datenstroms. Aus diesem Grund kann an dieser Stelle auf weitere Details zu diesem Thema verzichtet werden.

3.7 Sicherheitsbetrachtung

Bei dem Protokoll MQTT liegt das Hauptaugenmerk auf eine ressourcenschonende Option zum Informationsaustausch zwischen einzelnen IoT-Geräten. Selbst auf der offiziellen Homepage von MQTT wird das Thema Sicherheit sehr vorsichtig formuliert.

You can pass a user name and password with an MQTT packet in V3.1 of the protocol. Encryption across the network can be handled with SSL, independently of the MQTT protocol itself (it is worth noting that SSL is not the lightest of protocols, and does add significant network overhead). Additional security can be added by an application encrypting data that it sends and receives, but this is not something built-in to the protocol, in order to keep it simple and lightweight.

[Originaler Auszug <https://mqtt.org/faq/> unter dem Punkt "Does MQTT support security?" 08.12.2025]

Es wird prinzipiell die Möglichkeit zur Einbindung einer SSL-Verschlüsselung ermöglicht, jedoch muss diese vom Nutzer erst aktiviert werden. Zeitgleich wird darauf hingewiesen, dass der Einsatz dieser Verschlüsselung entgegen des Hauptvorteils von MQTT wirkt und somit der Einsatz mit bedacht gewählt werden soll.

Neben der Möglichkeit zum Einsatz von Algorithmen zur Verschlüsselung der Nachrichten lassen sich zusätzlich Nutzerkennungen mitsamt entsprechender Passwörter hinterlegen. Auf diese Weise soll ein unberechtigter Lesezugriff bei versandten Nachrichten verhindert⁸ werden.

4. Implementierung

Bedingt durch die Entwicklung einer eigenen Anwendung zum Tunneln von Daten via MQTT bietet sich der Einsatz einer TUN⁹-Device an. Ein TUN-Gerät ist eine spezifische Art von

⁸ oder zumindest erschwert

⁹ Akronym TUN in TUN-Device steht für TUNnel

virtuellem Netzwerkgerät im Linux-Kernel, das zur Implementierung von Netzwerk-Tunneln verwendet wird, insbesondere für VPN-Dienste (Virtual Private Network). Es arbeitet auf der Netzwerkschicht (Schicht 3) und verarbeitet IP-Pakete. [19].

Bei einem TUN-Device handelt es sich um eine virtuelle Netzwerkschnittstelle und kann vereinfacht als Punkt-zu-Punkt- oder Ethernet-Gerät betrachtet werden. Die Besonderheit liegt darin, dass anstelle einer Datenübertragung per physischen Medium, die Datenverbindung virtuell über eine eigene Anwendung aufgebaut wird. [19]

Der Kernel behandelt die Datenpakete der TUN-Device genauso, als würden diese von einem echten physischen Gerät abstammen. Die TUN-Device übernimmt dabei sämtliche Netzwerkrelevanten Aufgaben wie z.B. auch die Segmentierung des ankommenden Datenstroms. Aus diesem Grund kann an dieser Stelle auf weitere Details zu diesem Thema verzichtet werden.

4.1 Entwicklungsumgebung

Die Implementierung und Erprobung des MQTT-basierten Tunnels erfolgte in einer virtualisierten Testumgebung auf Basis von Oracle VirtualBox. Hierzu wurden zwei virtuelle Maschinen mit Debian Linux eingesetzt, die identisch konfiguriert wurden.

Die virtuellen Maschinen wurden in VirtualBox unter den Namen **mqtt-vm-a** und **mqtt-vm-b** angelegt und betrieben.

Beide virtuellen Maschinen verfügen über zwei Netzwerkschnittstellen. Eine Schnittstelle ist als NAT-Interface ausgeführt und ermöglicht den Zugriff auf externe Netzwerke. Die zweite Schnittstelle ist als Host-Only-Netzwerk konfiguriert und dient der direkten Kommunikation zwischen den virtuellen Maschinen sowie dem Hostsystem.

Die IP-Adressierung im Host-Only-Netzwerk ist statisch konfiguriert:

- **mqtt-vm-a**: 192.168.56.101
- **mqtt-vm-b**: 192.168.56.102

Der administrative Zugriff auf die Systeme erfolgt über SSH.

Für die Durchführung der Tests wurden folgende Benutzerkonten verwendet:

- **mqtt-vm-a**: Benutzer usera
- **mqtt-vm-b**: Benutzer userb

Schritt 1: Grundlegende Build- und Systemwerkzeuge

```
sudo apt update
sudo apt install -y \
  build-essential \
  cmake \
  git \
  pkg-config \
  libssl-dev \
  libgtest-dev \
  libcereal-dev \
  libspdlog-dev
```

Schritt 2: Google Test kompilieren und installieren

```
cd /usr/src/googletest
sudo cmake -S . -B build
sudo cmake --build build -j$(nproc)
sudo cmake --install build
sudo ldconfig
```

Schritt 3: Eclipse Paho MQTT (C-Bibliothek) installieren

```
cd ~
git clone https://github.com/eclipse/paho.mqtt.c.git
cd paho.mqtt.c
cmake -S . -B build -DPAHO_WITH_SSL=ON -DPAHO_BUILD_SHARED=ON
cmake --build build -j$(nproc)
sudo cmake --install build
sudo ldconfig
```

Schritt 4: Eclipse Paho MQTT (C++-Wrapper) installieren

```
cd ~
git clone https://github.com/eclipse/paho.mqtt.cpp.git
cd paho.mqtt.cpp
cmake -S . -B build
cmake --build build -j$(nproc)
sudo cmake --install build
sudo ldconfig
```

Schritt 5: Projekt kompilieren

```
cd ~/MQTT_Tunnel
mkdir -p build
cd build
cmake ..
make -j$(nproc)
```

4.2 Aufbau der Software

Die entwickelte Software zum Tunneln der TCP-Pakete über das MQTT-Protokoll gliedert sich in mehrere Komponenten. Diese einzelnen Softwarekomponenten gliedern sich dabei gemäß dem in Abbildung 4 gezeigten Aufbau. Die sprechenden Namen einzelner Dateien dienen der besseren Übersicht und bieten einen ersten Ausblick über die dort hinterlegte Funktion.

In der Datei *helpers.h* ist eine Hashfunktion zur Generierung von SHA256 Schlüsseln hinterlegt. Neben der Integrität der versandten Daten, wird hiermit zeitgleich die Funktionsfähigkeit der gesamten Applikation überprüft. Der ankommende Datenstroms aus der Datenquelle muss, trotz der mehrfachen Übersetzung in die unterschiedlichen Protokolle, mit dem Ankommenden an der Datensenke übereinstimmen.

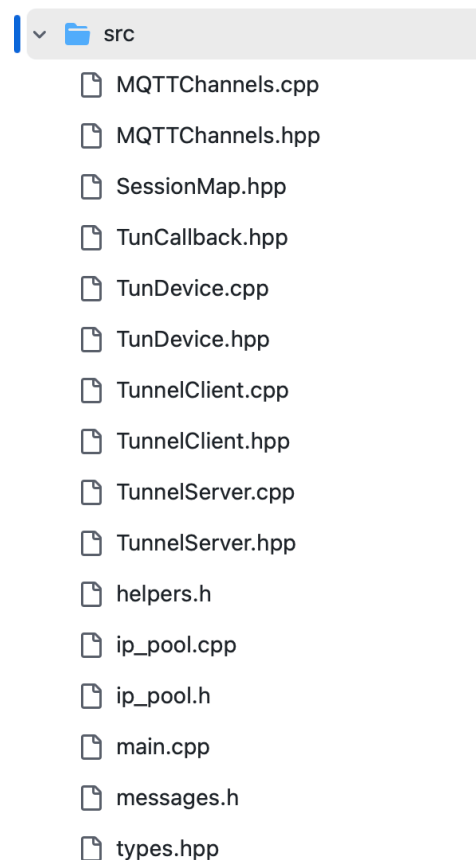


Abbildung 4 Auszug der Sourcefiles aus dem Github Repository¹⁰

Die Dateien *ip_pool.h* und *ip_pool.cpp* halten die Klassen für das Management des potentiellen IP-Pools bereit. Beim aktuellen Softwarestand wird ein IP-Bereich /24 unterstützt, was einer Höchstgrenze von 256 Teilnehmern entspricht.

¹⁰ https://github.com/PerHuepenbecker/MQTT_Tunnel/tree/main/src

4.3 Installation

Um die Anwendung auf dem lokalen System betreiben zu können, müssen die folgenden Schritte durchgeführt werden.

1. Installieren der benötigten Pakete
2. Klonen des git repository auf einen Lokalen Zweig
3. Navigation der Shell in das zuvor geklonte Repository
4. Ausführen des Befehls `mkdir build && cd build`
5. Ausführen des Befehls `cmake ..`
6. Ausführen des Befehls `make`

Die zuvor erwähnten Abhängigkeiten bzw. benötigten Pakete wie folgt.

- paho-MQTT C++
- GTest
- OpenSSL
- libsodium
- cereal
- spdlog
- fmt

Anschließend lässt sich die Anwendung über den Befehl aufrufen. Mittels des Parameters [Arguments] können verschiedene Betriebsmodi festgelegt

```
sudo ./mqtt_tunnel [ARGUMENTS]
```

werden. Die möglichen Argumente mit samt den dazugehörigen Erläuterungen sind der folgenden Tabelle aufgeführt. Dabei werden Parameter wie Betriebsmodus, Adresse des MQTT-Brokers, eine Client ID, MQTT-Topics oder das Versenden verschlüsselter Nachrichten eingestellt, bzw. zu- oder abgeschaltet.

Ein möglicher Befehl kann demnach folgendermaßen aussehen.

```
./mqtt_tunnel -m server -b tcp://192.168.0.2:1883 -c tunnelServer -C tunnelServerCommand -e
```

Um einen korrekten Betrieb zu gewährleisten, muss der ausführende User über erweiterte Berechtigungen verfügen.

Argument	Type	Flag	Required	Description
Operating Mode	<str>	-m, --mode	Yes	Defines the tunnel operation mode (client, server).
Broker	<url>	-b, --broker	Yes	The MQTT broker URI (e.g., tcp://localhost:8883).
Client ID	<str>	-i, --client-id	Yes	Identifier
Command Channel	<str>	-c, --command-channel	Yes	The MQTT topic used for tunnel setup
Verbose Mode	<flag>	-v, --verbose	No	Verbose Mode for deeper insights.
Encryption	<flag>	-x, --insecure	No	No Encryption
Server-Auth	<flag>	-t --ignore-server-auth	No	Client skips Public Key Identification of the server

4.4 TUN-Device-Anbindung

4.5 MQTT-Integration

4.6 Programmablauf

5. Tests und Analyse

5.1 Funktionale Tests

5.2 Performance und Latenz

5.3 Wireshark-Analyse

5.4 Fehlerfälle

6. Fazit

Abbildungsverzeichnis

Abbildung 1 Übersicht MQTT-Clients und MQTT-Broker	9
Abbildung 2 Darstellung des angestrebten Funktionsablaufs	10
Abbildung 3 Übersicht zum Informationsfluss der MQTT-Topics	10
Abbildung 4 Auszug der Sourcefiles aus dem Github Repository	17

Quellenverzeichnis

1. https://docs.oracle.com/cd/A97335_02/apps.102/a83722/overvie3.htm#1014532
2. <https://www.ibm.com/docs/en/app-connect/11.0.0?topic=corba-common-object-request-broker-architecture>
3. https://refubium.fu-berlin.de/bitstream/handle/fub188/4102/12_chap12.pdf
4. <https://learn.microsoft.com/de-de/dotnet/standard/native-interop/type-marshalling>
5. https://www-ois-com.translate.goog/index.php/about-corba/?_x_tr_sl=en&_x_tr_tl=de&_x_tr_hl=de&_x_tr_pto=sge#::~:~:text=CORBA%20is%20the%20world's%20leading,programming%20languages%2C%20and%20operating%20systems.
6. [https://www.elpassion.com/de/glossary/what-is-common-object-request-broker-architecture-corba#:~:text=Eine%20der%20Hauptfunktionen%20von%20CORBA%20\(%20Common,zwischen%20verschiedenen%20Plattformen%20und%20Programmiersprachen%20zu%20unterst%C3%BCtzen.](https://www.elpassion.com/de/glossary/what-is-common-object-request-broker-architecture-corba#:~:text=Eine%20der%20Hauptfunktionen%20von%20CORBA%20(%20Common,zwischen%20verschiedenen%20Plattformen%20und%20Programmiersprachen%20zu%20unterst%C3%BCtzen.)
7. <https://www.omg.org/corba/faq.htm>
8. <https://hilfe.comarch.de/cee/index.php/documentation/corba-schnittstelle/>
9. <https://www.arl.wustl.edu/Publications/2000-04/comp00fgk.pdf>
10. <https://umu.diva-portal.org/smash/get/diva2:1987951/FULLTEXT01.pdf>
11. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
12. <https://iot.telekom.com/de/blog/mqtt-protokoll-funktionsweise-anwendungen-und-vorteile-im-iot>
13. <https://www.opc-router.de/was-ist-mqtt/>
14. https://aws-amazon-com.translate.goog/what-is/mqtt/?_x_tr_sl=en&_x_tr_tl=de&_x_tr_hl=de&_x_tr_pto=sge#::~:~:text=Smart%20sensors%2C%20wearables%2C%20and%20other,can%20communicate%20IoT%20data%20efficiently.
15. <https://www.all-electronics.de/automatisierung/basiswissen-mqtt-was-kann-das-iot-kommunikations-protokoll/700581#:~:text=Antwort%205:%20MQTT%20verwendet%20QoS%2DModi%2C%20um%20die,QoS%2DStufen:%200%2C%201%20und%202%2C%20die%20verschiedene>
16. <https://www.pubnub.com/blog/what-is-mqtt-use-cases/>
17. <https://www.ibm.com/docs/de/ibm-mq/9.3.x?topic=overview-mq-telemetry-transport-protocol>
18. <http://www.bbs-1.de/bbs1/umat/netze/netz8.html>
19. <https://www.kernel.org/doc/html/latest/networking/tuntap.html>
20. https://www.researchgate.net/publication/317391853_IoT_real_time_data_acquisition_using_MQTT_protocol
21. [Evaluation of MQTT Bridge Architectures in a Cross-Organizational Context von Keila Lima, Tosin Daniel Oyetoyan, Rogardt Heldal, Wilhelm Hasselbring aus 01/2025](#)
22. <https://www.emqx.com/en/blog/mqtt5-new-features-payload-format-indicator-and-content-type>
23. <https://mymqtt.app/de/mqtt-qos-levels>
- 24.